TRADEGRAPH

Agentic Financial Research & Intelligence Platform

Final Production Blueprint

RAG + Hybrid Retrieval + Quantitative Analysis + LangChain + LangGraph + Evaluation + Deployment

Purpose

Build a production-deployed financial research system that investigates market questions using financial documents, market data, hybrid retrieval, quantitative tools, iterative agentic research, evidence tracing, and citation-backed synthesis.

Dimension	Final decision
Product	Financial Research & Decision-Support Platform — not an autonomous trading bot
Primary workflow	Goal → plan → research → retrieve → evaluate → research again if needed → synthesize → verify
Primary vector DB	Qdrant
Agent orchestration	LangGraph
LLM/RAG framework	LangChain
Observability	LangSmith + OpenTelemetry
Deployment	Docker + managed cloud services / container platform

Design principle: every technology must solve a real architectural problem. Do not add technologies merely to increase the stack.

1. Executive Architecture

The final system is a stateful research platform rather than a query → retrieve → generate chatbot. The original ResearchGraph specification defines this distinction and emphasizes explicit state, evidence traceability, iterative research, and verification. ■filecite■turn1file4■L503-L526■

```
Internet/User
↓
React + TypeScript + Tailwind
↓ HTTPS / SSE
FastAPI API + Auth + Rate Limits
↓
PostgreSQL + Redis + Object Storage
↓
Research Worker → LangGraph
↓
Planner → Query Decomposer → Parallel Research
↓
SEC / Filings RAG + News RAG + Market Data + Quant Tools
↓
Dense Retrieval + BM25 → Reciprocal Rank Fusion → Reranker
↓
Evidence Extraction → Claim/Evidence Graph → Verification
↓
Gap Detection → Research Again OR Synthesis
↓
Critic → Citation Validator → Final Report
↓
PostgreSQL → API → Research Dashboard
```

The source architecture similarly separates UI, API, LangGraph orchestration, LLM/RAG, tools, PostgreSQL/Qdrant/Redis data services, and observability. ■filecite■turn1file0■L15-L41■

2. Product Scope

Core use case

A user asks a complex financial research question. TradeGraph decomposes it, retrieves relevant evidence, performs quantitative analysis where required, detects evidence gaps or contradictions, researches further when necessary, and produces a traceable report.

Example questions

- Why did NVIDIA outperform the S&P; 500 after its latest earnings event?
- What were the major drivers of Apple's margin changes across recent filings?
- How did a company's earnings surprises historically relate to short-term market reaction?
- Compare the fundamental and market-risk profiles of two companies.
- What evidence supports and contradicts a particular investment hypothesis?

Product boundary

- **Do:** research, retrieve, calculate, compare, explain, cite, expose uncertainty, and support reproducible analysis.
- **Do not:** present guaranteed returns, autonomously execute trades, or masquerade as a personalized investment adviser.
- Backtesting is an analytical capability; it must include out-of-sample evaluation, transaction costs, slippage assumptions, and explicit warnings about historical-data limitations.

3. Final Industry-Oriented Technology Stack

Layer	Technology	Purpose
Language	Python	LLM, RAG, evaluation, data and quantitative ecosystem
Frontend	React + TypeScript + Tailwind	Research workspace, progress, evidence, charts
API	FastAPI	Async API, auth, streaming, job management
LLM framework	LangChain	Models, prompts, tools, retrievers, structured output
Agent workflow	LangGraph	State, branching, loops, checkpoints, controlled orchestration
LLM	Provider abstraction	Keep model layer replaceable
Embeddings	BGE / API embedding model	Dense semantic retrieval
Vector DB	Qdrant	Vector search + metadata filtering
Sparse retrieval	BM25	Exact names, tickers, identifiers, financial terminology
Reranker	BGE / equivalent	Second-stage relevance ranking
RDBMS	PostgreSQL	Users, jobs, documents, claims, evidence, reports, audits
Cache/queue	Redis	Caching, rate limits, background jobs, transient state
Raw storage	S3-compatible object storage	Original filings, PDFs, processed artifacts
Quant	Polars/Pandas + NumPy	Returns, volatility, event studies, backtesting
Observability	LangSmith + OpenTelemetry	LLM/graph traces + application traces
Containers	Docker	Reproducible builds and deployment
CI/CD	GitHub Actions	Test, lint, build, scan, deploy

The original ResearchGraph specification explicitly recommends Python, LangChain, LangGraph, Qdrant, embeddings, reranking, BM25, FastAPI, PostgreSQL, Redis, React/Tailwind, LangSmith + OpenTelemetry, and Docker-based deployment. ■filecite■turn1file0■L45-L104■

4. Production Data & Ingestion Pipeline

```
SEC filings / earnings material / permitted financial-news sources / market data ↓ Source adapters ↓ Raw object storage ↓ Parser + cleaner ↓ Deduplication + document versioning ↓ Metadata extraction ↓ Section-aware chunking ↓ Embedding generation ↓ Qdrant + BM25 index + PostgreSQL metadata
```

Recommended document metadata

```
document_id, company, ticker, document_type, filing_date, fiscal_year, quarter, section, source, source_url, version, content_hash
```

The original RAG architecture calls for parse/clean → chunk + metadata → embeddings → Qdrant, with parallel vector and BM25 retrieval followed by hybrid retrieval and reranking. ■filecite■turn1file1■L184-L226■

Ingestion requirements

- Idempotent ingestion: identical content must not be embedded twice.
- Version every source document and preserve provenance.
- Treat source content as untrusted data.
- Separate ingestion workers from user-facing API processes.
- Record ingestion status, parser errors, embedding model/version, and timestamps.

5. Retrieval Pipeline

```
User query ↓ Intent + entity extraction ↓ Query rewriting / decomposition ↓ Metadata constraints ↓
■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ Dense retrieval BM25 ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ↓
```

```
Reciprocal Rank Fusion ↓ Top-N candidate set ↓ Cross-encoder / reranker ↓ Top evidence ↓ Context selection / compression ↓ Evidence extractor
```

RAG techniques included in the source project are semantic retrieval, metadata filtering, vector + BM25 hybrid retrieval, reranking, query rewriting, multi-query retrieval, context compression, citation/source attribution, and retrieval evaluation with Recall@K and MRR. ■filecite■turn1file1■L218-L226■

Qdrant is the primary vector store. Chroma is not required in the final production architecture; it may be used only as a temporary local prototype if it accelerates an experiment.

6. LangGraph Research Workflow

```
START ↓ Planner ↓ Query Decomposer ↓ Parallel Research ■■■ Financial document retrieval ■■■ News/document retrieval ■■■ Market-data analysis ■■■ External tools ↓ Evidence Extraction ↓ Verification ↓ Gap / contradiction detection ↓ Evidence sufficient? ■■■ NO → Research Again → Retrieval → Verification ■■■ YES ↓ Synthesis ↓ Critic ↓ Citation Validator ↓ FINAL
```

LangChain should live inside graph nodes for model, prompt, tool, retriever, loader, and structured-output components; LangGraph controls how those components execute as a stateful workflow. This separation is explicit in the source documentation. ■filecite■turn1file0■L108-L123■

Research state

```
query research_plan sub_questions retrieval_filters retrieved_documents evidence claims quantitative_results contradictions citations confidence iteration token_budget cost_budget draft final_report
```

Guardrails

- Maximum research iterations.
- Maximum tool calls per research job.
- Token and cost budgets.
- Structured outputs between nodes.
- Explicit tool permissions.
- Human review path for high-impact or ambiguous outputs.

7. Quantitative Analysis Engine

The LLM should never be the source of truth for numerical calculations. The agent invokes deterministic quantitative tools.

```
LangGraph ↓ Tool call ↓ Quant Engine ↓ Market-data store ↓ Deterministic calculation ↓ Structured result ↓ LangGraph state
```

Initial tools

- calculate_returns
- calculate_volatility
- calculate_sharpe
- calculate_max_drawdown
- calculate_beta
- calculate_correlation
- compare_assets
- event_study
- backtest_strategy

Event study example

For an earnings event, calculate pre-event baseline and post-event windows such as +1, +3, +5, and +20 trading days, including absolute and benchmark-adjusted returns and volume/volatility changes.

Backtesting requirements

- Train/in-sample vs out-of-sample separation.
- Walk-forward validation where appropriate.
- Transaction costs and slippage assumptions.
- No look-ahead bias.
- No survivorship-bias shortcuts where avoidable.
- Clearly distinguish historical analysis from future prediction.

8. Evidence, Claims & Citation Architecture

Do not generate citations after writing the report. Build the evidence chain first.

```
Source ↓ Passage / chunk ↓ Evidence item ↓ Claim ↓ Citation ↓ Synthesis
```

Every important claim should have machine-readable provenance: document ID, source, section, timestamp/version, supporting passage, and claim ID.

The source specification emphasizes that important conclusions should be connected to evidence and that the final report should support traceability. ■filecite■turn1file4■L520-L526■

Citation validator

```
Generated claim ↓ Retrieve cited evidence ↓ Does evidence entail/support the claim? ■■■ YES → accept ■■■
NO → rewrite / remove / flag
```

9. Caching & Prompt Engineering

Redis cache layers

Cache	Key basis	Rule
Embeddings	text + embedding model/version	Invalidate when embedding model changes
Retrieval	query + filters + retrieval config/version	Invalidate when retrieval configuration changes
LLM	model + prompt version + input + generation settings	Never reuse stale results across incompatible versions
Job state	research_job_id	Transient workflow state only

Prompt repository

```
prompts/ ■■■ planner/ ■■■ query_rewriter/ ■■■ evidence_extractor/ ■■■ verifier/ ■■■ contradiction/
■■■ synthesizer/ ■■■ critic/ ■■■ citation_validator/
```

Record model, prompt version, temperature/generation settings, token usage, latency, and cost for every important LLM call.

10. Evaluation & Experimentation

Evaluation is a first-class subsystem, not a final demo step.

Area	Metrics / checks
Retrieval	Recall@K, Precision@K, MRR, nDCG
RAG	Context relevance, answer relevance, faithfulness
Citations	Citation correctness, citation completeness, claim-support rate

Area	Metrics / checks
Agent	Tool-selection accuracy, unnecessary calls, research completeness, iteration count
Quant	Sharpe, drawdown, turnover, transaction costs, out-of-sample performance
System	Latency, token usage, cost, error rate, cache hit rate

Core ablation experiments

- Dense retrieval vs BM25 vs hybrid retrieval.
- Hybrid retrieval vs hybrid + reranker.
- No metadata filtering vs metadata filtering.
- No query rewriting vs query rewriting.
- Single-shot RAG vs iterative LangGraph research.
- Prompt v1 vs prompt v2.
- No cache vs cache.

Create a benchmark dataset before optimizing. Keep the benchmark fixed so architecture changes can be compared fairly.

11. LangSmith + OpenTelemetry

LangSmith is the AI/agent observability layer; OpenTelemetry is the broader application/infrastructure tracing layer.

LangSmith	OpenTelemetry
LLM traces	API traces
LangChain/LangGraph execution	Worker traces
Prompt/version visibility	DB/Redis latency
Tool calls	Infrastructure health
RAG/evaluation runs	Cross-service trace propagation
Agent debugging	Production failure analysis

Every research request should carry a research_id / trace_id through API → worker → graph → tools → final report.

The source stack explicitly recommends LangSmith + OpenTelemetry for tracing LLM calls, graph execution, tools, latency, and failures. ■filecite■turn1file0■L98-L104■

12. Security, Reliability & Financial Safety

- Treat all retrieved documents and external text as untrusted data; never let them override system instructions.
- Keep tool permissions narrow and explicit.
- Validate all external inputs and tool arguments.
- Enforce maximum iterations, tool calls, token budgets, and cost budgets.
- Use authentication and authorization for research history and private documents.
- Apply rate limiting and abuse controls.
- Store secrets only in managed secret/environment configuration.
- Audit major tool calls and state transitions.
- Expose evidence gaps and uncertainty instead of forcing a conclusion.

- Do not connect live brokerage execution in the initial product.
- Clearly label research as informational/decision-support output rather than guaranteed financial advice.

The original project specifically requires untrusted-document handling, source attribution, narrow tool permissions, iteration/cost limits, structured outputs, logging, separation of content from instructions, human review for sensitive outputs, and explicit uncertainty. ■filecite■turn1file3■L437-L446■

13. Production Deployment

```
Internet ↓ Cloudflare/CDN ↓ React Frontend ↓ HTTPS FastAPI API ↓ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ↓ ↓
↓ PostgreSQL Redis Qdrant Cloud ■ ■ ■ ↓ ■ Research Worker ■ ↓ ■ LangGraph ■ ↓ ■■■■■■■■■■→ Object Storage
```

Recommended first deployment split

Component	Deployment
Frontend	Vercel or equivalent managed frontend platform
FastAPI	Docker container on Railway/Render/Fly.io/AWS-style container platform
Research worker	Separate Docker container/service
PostgreSQL	Managed PostgreSQL
Redis	Managed Redis
Qdrant	Qdrant Cloud
Object storage	S3-compatible bucket
CI/CD	GitHub Actions

Start with managed services and Dockerized application/worker services. Kubernetes is deliberately deferred until there is a demonstrated scaling requirement.

14. CI/CD & Environments

```
git push ↓ GitHub Actions ■■■ Unit tests ■■■ Integration tests ■■■ Lint / format ■■■ Type checks ■■■
Evaluation smoke tests ■■■ Dependency/security scan ■■■ Docker build ■■■ Deploy
```

Use separate development, staging, and production environments. Retrieval configurations and prompts should be versioned so experiments do not silently alter production behavior.

15. Final Repository Structure

```
tradegraph/ ■■■ apps/ ■ ■■■ api/ # FastAPI ■ ■■■ web/ # React + TypeScript ■■■ src/ ■ ■■■ graph/ #
LangGraph state/nodes/edges ■ ■■■ agents/ # agent policies ■ ■■■ tools/ # SEC/news/market/quant tools ■
■■■ rag/ ■ ■ ■■■ ingestion/ ■ ■ ■■■ chunking/ ■ ■ ■■■ embeddings/ ■ ■ ■■■ vector/ ■ ■ ■■■ bm25/ ■ ■
■■■ hybrid/ ■ ■ ■■■ reranking/ ■ ■ ■■■ citations/ ■ ■■■ quant/ ■ ■■■ prompts/ ■ ■■■ models/ ■ ■■■
cache/ ■ ■■■ evaluation/ ■ ■■■ observability/ ■■■ data/ ■ ■■■ benchmark/ ■ ■■■ samples/ ■■■
migrations/ ■■■ tests/ ■■■ docker/ ■■■ .github/workflows/ ■■■ .env.example ■■■ pyproject.toml ■■■
README.md
```

This extends the source project's recommended organization of apps, graph, agents, tools, RAG, models, evaluation, observability, data, tests, Docker, and environment configuration. ■filecite■turn1file3■L408-L436■

16. Build Roadmap — V1 to V5

Phase	Build	Exit criterion
V1	Financial corpus + ingestion + Qdrant + BM25 + basic RAG + citations	Cited financial research report works end-to-end
V2	LangChain tools + LangGraph planner/research/verify loop	Agent can research and retry when evidence is insufficient
V3	Market-data tools + event studies + comparative quantitative analysis	Research reports combine retrieved evidence with deterministic calc

Phase	Build	Exit criterion
V4	Redis caching + prompt versioning + LangSmith + OTel + auth + rate limits + scheduling + worker pool	Production-quality deployed application with traces and controls
V5	Evidence graph + contradiction detection + benchmark suite + backtesting + dashboard	Advanced system with measurable retrieval/agent/quant improvements

The original learning roadmap follows the same progression: basic RAG → production RAG → LangChain research assistant → LangGraph workflow → agentic research → UI/API → evaluation → production.
■filecite■turn1file3■L374-L404■

17. Master Checklist

Architecture

- [ ] Clear service boundaries
- [ ] Async research jobs
- [ ] Persistent research state
- [ ] Versioned documents
- [ ] Evidence provenance

RAG

- [ ] Chunking
- [ ] Embeddings
- [ ] Qdrant
- [ ] Metadata filtering
- [ ] BM25
- [ ] Hybrid retrieval
- [ ] Reranker
- [ ] Query rewriting
- [ ] Context compression
- [ ] Citation validation

Agentic

- [ ] LangChain tools
- [ ] LangGraph state
- [ ] Planner
- [ ] Parallel research
- [ ] Verification
- [ ] Gap detection
- [ ] Retry loop
- [ ] Critic
- [ ] Explicit stopping criteria

Quant

- [ ] Returns
- [ ] Volatility

- [ ] Sharpe
- [ ] Drawdown
- [ ] Beta/correlation
- [ ] Event study
- [ ] Backtesting
- [ ] Out-of-sample evaluation
- [ ] Costs/slippage

Production

- [ ] Redis
- [ ] PostgreSQL
- [ ] Object storage
- [ ] Authentication
- [ ] Rate limiting
- [ ] Prompt versioning
- [ ] Cost tracking
- [ ] LangSmith
- [ ] OpenTelemetry
- [ ] Docker
- [ ] CI/CD
- [ ] Staging
- [ ] Production deployment

Evaluation

- [ ] Benchmark dataset
- [ ] Recall@K
- [ ] MRR
- [ ] nDCG
- [ ] Citation correctness
- [ ] Faithfulness
- [ ] Agent trajectory evaluation
- [ ] Latency
- [ ] Cost
- [ ] Cache hit rate
- [ ] Quant backtest metrics

18. MVP Definition

Do not build the entire platform first. The first useful release should be small and demonstrable.

```
User question ↓ Planner creates 3–5 subquestions ↓ Financial document corpus ↓ Hybrid retrieval ↓
Reranking ↓ Evidence extraction ↓ Verification ↓ Cited report
```

MVP success criterion

Given a financial research question, TradeGraph produces a useful report whose important claims can be traced to retrieved evidence and whose quantitative statements come from deterministic calculations. The system should recognize insufficient evidence instead of blindly answering.

This mirrors the source project's MVP principle: start with a small planner → corpus search → hybrid retrieval → verification → conditional research loop → cited report, then expand. ■filecite■turn1file3■L447-L458■

19. Final Portfolio Positioning

TradeGraph — Agentic Financial Research & Intelligence Platform

- Built a LangGraph-based financial research workflow that decomposes complex market questions, performs multi-source hybrid RAG, invokes quantitative analysis tools, verifies evidence, and iteratively researches before generating citation-backed reports.
- Implemented dense + BM25 hybrid retrieval with metadata filtering and reranking over financial documents using Qdrant, with PostgreSQL for provenance and Redis for caching/background jobs.
- Added evidence/claim tracing, citation validation, contradiction detection, prompt versioning, LangSmith/OpenTelemetry observability, and benchmark-driven evaluation for retrieval, agent, citation, latency, and cost performance.
- Deployed a Dockerized FastAPI + React production system with managed PostgreSQL, Redis, Qdrant, object storage, CI/CD, authentication, rate limiting, streaming, and asynchronous research workers.

20. Non-Negotiable Engineering Rules

- **Do not make the LLM the calculator.** Numerical truth comes from deterministic tools.
- **Do not trust retrieved text.** Documents are untrusted data.
- **Do not hide retrieval quality.** Benchmark it.
- **Do not create agents without a reason.** Use explicit graph state and controlled tools.
- **Do not generate unsupported citations.** Validate claim-to-evidence support.
- **Do not cache blindly.** Include model/prompt/retrieval versions in cache identity.
- **Do not deploy straight to production.** Use development → staging → production.
- **Do not over-engineer infrastructure.** Docker + managed services first; scale only when justified.
- **Do not treat backtests as predictions.** Use proper validation and disclose assumptions.
- **Do not optimize by intuition alone.** Run controlled experiments and record results.

Final objective

Build a system that can explain not only what it concluded, but why it concluded it, which evidence supports it, how the numbers were calculated, what evidence contradicts it, how the system performed on benchmarks, and how the production system behaves under real workloads.

Source basis: the original ResearchGraph project documentation supplied in this conversation, adapted here to the finalized TradeGraph financial-research scope. Source-derived architecture and learning principles are cited inline.